ConstRSA: A Minimal Constant-Time RSA-512 Digital Signature with Statistical Timing-Leak Verification on Standard Laptops

Maleesha Rathnayake
2020/ICT/47

Department of Physical Science
University of Vavuniya, Sri Lanka

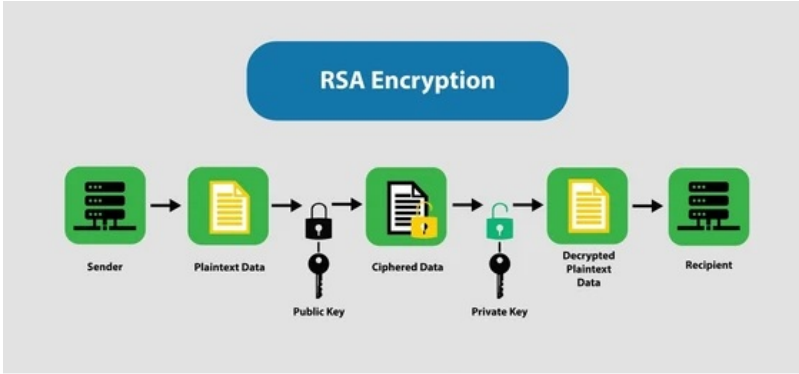
January 2026

Presentation Outline

- Background
- Problem Definition
- Aim & Research Questions
- Objectives
- Methodology (Implementation + Testing)
- Dataset & Welch's t-test
- Expected Outcomes & Work Plan

Background: Digital Signatures

- A **digital signature** provides:
  - **Authentication**: who signed it
  - **Integrity**: data not changed
  - **Non-repudiation**: signer cannot deny easily
- Common signature algorithms: **RSA**, **DSA**, **ECDSA**
- RSA = **Rivest–Shamir–Adleman**



Example: signing with private key and verifying with public key

Why Constant-Time Matters

- Side-channel attacks can use **timing** or **CPU hardware behavior** (cache, branch predictor).
- If signing time depends on secret key bits, attackers may infer the key using many measurements.
- **Constant-time** goal: execution time and memory access should not depend on secret data.

Problem Definition

- No small, easy-to-audit **constant-time RSA signature** example for student/research testing.
- Existing implementations are often inside **large libraries** with complex optimizations.
- Hard to:
  - understand constant-time behavior,
  - test for timing leakage,
  - use for teaching/benchmarking.

Research Aim

Aim

Design, implement, and statistically validate a **minimal constant-time RSA-512 digital signature** (**RSA-PSS + SHA-256**) that can be tested on a normal laptop.

Note

RSA-512 is used here as an **academic / toy-size** parameter set for fast experiments, not for production security.

Research Questions

- **RQ1:** How to implement RSA-512 signing in C without secret-dependent branches and memory access?
- **RQ2:** Does the implementation show detectable timing leakage under fixed vs random testing?
- **RQ3:** What performance overhead is introduced by constant-time engineering compared to a baseline?

Objectives

Primary Objective

To design, implement, and statistically validate a minimal constant-time RSA-512 signature (RSA-PSS + SHA-256), targeting **under 10 KB** for core signing logic.

- Implement branchless signing using CRT, Montgomery multiplication, fixed-window exponentiation
- Remove secret-dependent branching and memory access patterns
- Collect timing traces and apply Welch's t-test (fixed vs random)
- Benchmark signing and verification performance
- Publish code and documentation as open-source

Methodology Overview

- **Build-and-test** approach:
  - Implement constant-time RSA-512 signature
  - Measure timing repeatedly on a laptop
  - Apply statistical testing to check leakage
- Constant-time rules:
  - no secret-dependent **if/else** branches
  - no secret-dependent **table lookups / memory access**

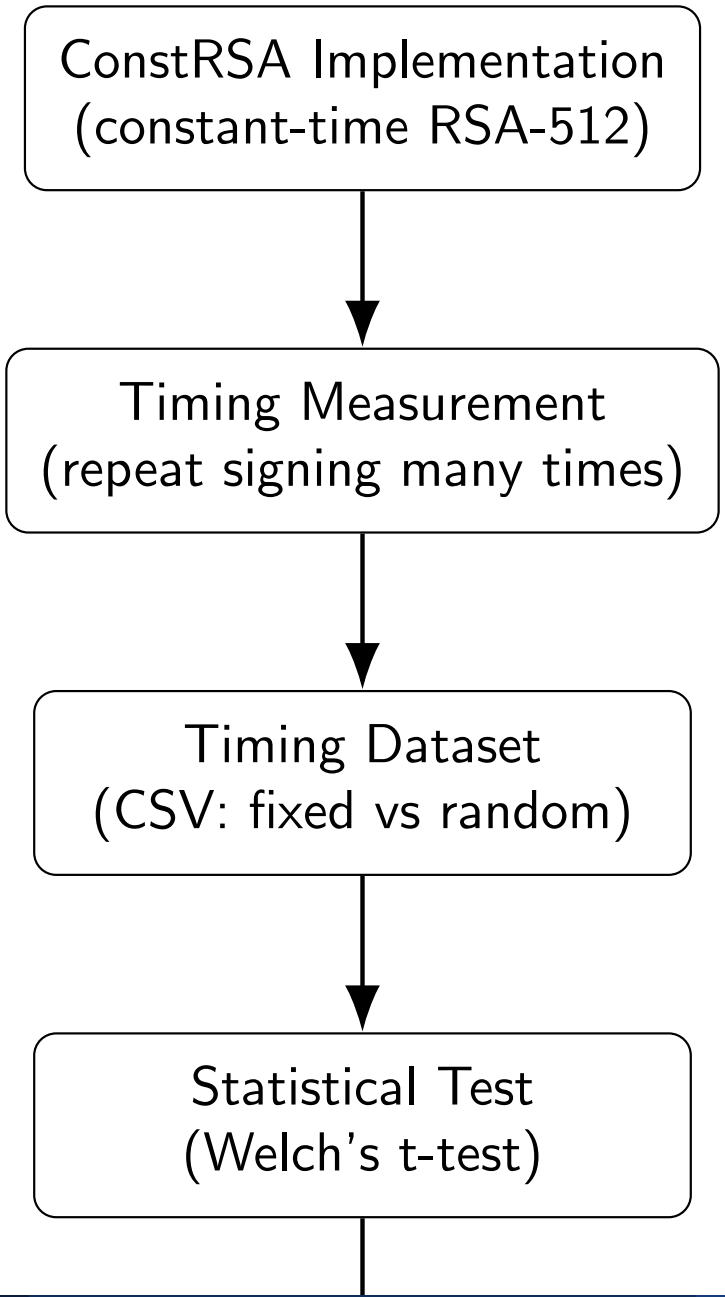
Dataset (Timing Traces)

- Dataset is a **timing dataset**.
- It will be created by running the signing function many times and recording time.
- Stored as **CSV**, with fields like:
  - run id, group (fixed/random), signing time (ns or CPU cycles)
- Used to compare timing behavior between fixed and random groups.

Leakage Testing (Welch's t-test)

- Two groups:
  - **Fixed**: same message repeated
  - **Random**: different random messages
- Record signing time for many runs in both groups
- Apply **Welch's t-test**:
  - If no significant difference, report **no detectable leakage under this test setup**

Workflow of the Proposed Study



Expected Outcomes

1. Minimal constant-time RSA-512 signature implementation (target: under 10 KB)
2. Statistical results from timing experiments (detectable leakage or not)
3. Open-source code + documentation + testing scripts
4. Clear example for secure coding and cryptography education

Work Plan (High Level)

- Literature review and design finalization
- Implementation (constant-time signing + verification)
- Timing data collection (fixed vs random)
- Statistical analysis (Welch's t-test)
- Benchmarking and final write-up
- Open-source release + submission

Thank You

Thank you.

Questions?